

Guillermo Román

guillermo.roman@upm.es

Lars-Åke Fredlund

lfredlund@fi.upm.es

Manuel Carro

mcarro@fi.upm.es

Julio García

juliomanuel.garcia@upm.es

Tonghong Li

tonghong@fi.upm.es

- Fechas de entrega y penalización:

Hasta el Martes 10 de noviembre, 16:00 horas	0
Hasta el Miércoles 11 de noviembre, 16:00 horas	20 %
Hasta el Jueves 21 12 noviembre, 16:00 horas	40 %

Después la puntuación máxima será 0
- Se comprobará plagio y se actuará sobre los detectados.
- Usad las horas de tutoría para preguntar sobre programación – son oportunidades excelentes para aprender.

Entrega

- Todos los ejercicios de laboratorio se deben entregar a través de
`http://deliverit.fi.upm.es`
- El/los fichero(s) que hay que subir es/son `Utils.java`.
- La clase debe estar en el paquete `aed.individual6`.
- La documentación de la API de `aedlib.jar` está disponible en
`http://costa.ls.fi.upm.es/teaching/aed/docs/aedlib/`

Tarea 1: Vértices Alcanzables

- Termina la implementación del método

```
static <V> Set<Vertex<V>>  
    reachableFromBoth(DirectedGraph<V, Boolean> g,  
                      Vertex<V> v1,  
                      Vertex<V> v2)
```

dentro el fichero `Utils.java`.

- El método debe devolver un conjunto que contenga los vértices alcanzables desde *ambos* vértices `v1` y `v2` en el grafo `g`. Es decir, si un vértice solo es alcanzable desde `v1` pero no desde `v2` no se debe incluir (y viceversa).
- **Notad:** para calcular los vertices alcanzables solo se puede seguir aristas cuyos elementos son `true`.

Conjuntos/sets en aedlib

- Un conjunto en aedlib implementa la interfaz:

```
public interface Set<E> extends Iterable<E> {  
    public boolean isEmpty();  
    public int size();  
  
    public boolean add(E elem);  
    public boolean remove(E elem);  
    public boolean contains(Object o);  
}
```

- Consultad la documentación de aedlib para más detalles.
- Hay dos clases que implementan la interfaz Set: HashMapSet (más eficiente) y PositionListSet (menos eficiente).

Tarea 2: Caminos Aceptables

- Termina la implementación del método

```
static <V> PositionList<Edge<Integer>>  
    existsPathLess(UndirectedGraph<V,Integer> g,  
                   Vertex<V> from, Vertex<V> to,  
                   int limit)
```

dentro el fichero Utils.java.

- El método debe devolver un camino (una secuencia de **aristas**) que lleva desde el vértice *from* al vértice *to*.
- *Importante*: solo se admite caminos donde la suma de los elementos de las aristas del camino sea $< \text{limit}$.
- Si no existe un camino *aceptable* entonces el método debe devolver *null*.

Notas Generales

- El proyecto debe compilar sin errores y debe cumplirse la especificación de los métodos a completar
- Debe pasar todos los test `TesterInd6` correctamente sin mensajes de error
- **Nota:** una ejecución sin mensajes de error y que pase todas las pruebas **no** significa que la implementación sea correcta (es decir, que funcione bien para cada posible entrada)
- Todos los ejercicios se corrigen manualmente antes de dar la nota final